

Módulo 8 – Capítulo 09 – Sección 01

Model registry local: Hugging Face Hub, MLflow y soluciones self-hosted

Un sistema de ML que no tiene un registro centralizado de sus modelos es un sistema que no puede ser operado por más de una persona sin caos. El model registry es la infraestructura que permite responder a las preguntas más básicas del gobierno de modelos en producción: ¿qué modelo está corriendo en producción ahora mismo?, ¿qué datos se usaron para entrenarlo?, ¿cómo era el modelo antes de la actualización del martes?, ¿quién tiene permiso de descargar los pesos del modelo fine-tuneado sobre datos de clientes? Sin un registry, estas preguntas se responden con conversaciones en Slack y archivos nombrados `modelo_final_v3_definitivo.gguf` en un disco compartido.

Hugging Face Hub es el registry de facto para el ecosistema de modelos open weights. Su arquitectura sobre Git+LFS permite versionado de archivos de pesos con historial de commits, diffs de metadata y reanudación automática de descargas interrumpidas. La API Python `huggingface_hub.HfApi` permite automatizar el ciclo de vida completo: `upload_folder(folder_path="./modelo", repo_id="org/modelo-v1")` publica una nueva versión del modelo; `model_info(repo_id="org/modelo-v1")` recupera la metadata; `list_models(author="org")` enumera todos los modelos de la organización. Para organizaciones con requisitos de privacidad que impiden usar el Hub público, Hugging Face Hub Enterprise ofrece una instancia privada detrás del firewall corporativo con los mismos controles de acceso y APIs.

MLflow Model Registry es la opción más integrada para equipos que ya usan MLflow para el tracking de experimentos de entrenamiento. Cada run de training registra automáticamente las métricas (loss, eval accuracy, golden set score), los hiperparámetros (learning rate, rank de LoRA, batch size) y los artefactos del modelo. La transición a producción ocurre a través del sistema de stages: un modelo se mueve de `None` a `Staging` cuando pasa los tests automatizados de calidad, y de `Staging` a `Production` con aprobación manual o tras el canary deployment exitoso. El historial completo de transiciones de stage queda registrado con timestamp y usuario responsable de la decisión, que es el log de

auditoría que los procesos de compliance requieren. La API `mlflow.register_model("runs:<run_id>/adapter", "modelo-soporte")` y `mlflow.pyfunc.load_model("models:/modelo-soporte/Production")` son los dos extremos del ciclo de registro y despliegue.

Las **soluciones self-hosted** son necesarias cuando los requisitos de privacidad o compliance impiden usar servicios externos para almacenar los pesos. MinIO como object storage compatible con S3, combinado con una capa de metadata personalizada (una tabla en PostgreSQL con los campos de registro de modelo), cubre los casos más básicos. Para organizaciones con mayor inversión en infraestructura de datos, Apache Atlas o DataHub pueden servir como registros de metadata para modelos ML, integrándose con el linaje de datos ya gestionado por la plataforma de datos corporativa. La complejidad de mantener un registry self-hosted completo es significativa; es frecuentemente más económico invertir en Hugging Face Hub Enterprise o MLflow desplegado en el cluster corporativo que construir y mantener la infraestructura desde cero.

Componentes de un model registry

- **Almacenamiento de artefactos:** pesos del modelo en object storage (S3, GCS, MinIO); el registry gestiona metadata y punteros a artefactos, no necesariamente el almacenamiento directo.
- **Versionado semántico:** cada versión identifica: commit SHA del código de entrenamiento, SHA del dataset, hiperparámetros, métricas de evaluación; permite reproducir y auditar cada versión.
- **Model card:** README.md en Hugging Face o YAML metadata en MLflow que describe el modelo, su tarea, los datos de entrenamiento, las métricas de evaluación, las limitaciones y las instrucciones de uso.
- **Control de acceso:** modelos privados en Hugging Face Hub con token de autenticación; MLflow usa permisos del object storage subyacente; para modelos con datos sensibles, control de acceso granular es requisito de compliance.
- **Búsqueda y descubrimiento:** filtros por tarea, arquitectura, idioma, fecha y métricas; Hugging Face Hub via `list_models()`; MLflow via búsqueda por tags y métricas registradas.

Nota del Arquitecto: El model registry no es un lujo que se añade cuando el equipo crece: es la infraestructura mínima para que el fine-tuning sea un proceso repetible y

auditable. El día que necesitas hacer un rollback a la versión anterior del modelo en producción a las 3am durante un incidente, descubrirás que "el modelo está en el servidor de producción en `/opt/models/`" no es una respuesta operacional suficiente.

El model registry es el componente que hace posible el ciclo de vida de los modelos como artefactos de ingeniería. La sección siguiente detalla las estrategias de versionado específicas para cada tipo de artefacto de modelo.

Módulo 8 – Capítulo 09 – Sección 02

Versionado de modelos: checkpoints, adaptadores LoRA y snapshots

El versionado de artefactos de modelo presenta complejidades que no existen en el versionado de software convencional: los archivos de pesos son de 2-80 GB, las diferencias entre versiones no son diffables con herramientas de texto como `git diff`, y el "artefacto" a versionar puede ser el modelo base completo, un adaptador LoRA de 50-300 MB, o la combinación de ambos fusionada en un snapshot. Tratar los tres con la misma estrategia de versionado produce confusión; cada uno tiene su estrategia óptima.

Los **checkpoints de entrenamiento** son capturas del estado completo del proceso de fine-tuning en un momento dado: incluyen los pesos del adaptador LoRA, el estado del optimizador Adam (los primeros y segundos momentos por parámetro), el step actual y la posición en el dataset. Su función es permitir reanudar el entrenamiento desde ese punto exacto si el proceso se interrumpe —por un fallo de hardware, una instancia Spot interrumpida o simplemente para evaluar el modelo a distintos puntos del entrenamiento. Hugging Face `Trainer` con `save_steps=100` guarda un checkpoint cada 100 pasos; `save_total_limit=3` mantiene solo los 3 más recientes para no saturar el almacenamiento. Los checkpoints son 2-3x más grandes que el adaptador LoRA solo (por el estado del optimizador incluido) y no deben confundirse con el artefacto de distribución final.

Los **adaptadores LoRA** son el artefacto de versionado más compacto y versátil: un archivo SafeTensors de 50-300 MB que contiene únicamente los pesos aprendidos por las matrices A y B de todas las capas de atención. Al ser independientes del modelo base, se versionan separadamente: el modelo base Llama 3.1 8B ocupa 16 GB y es idéntico para todos los proyectos que lo usan; el adaptador LoRA de 150 MB es específico de cada tarea y puede iterarse y versionarse frecuentemente sin replicar el modelo base. La convención de naming en Hugging Face Hub para adaptadores LoRA es `{org}/{modelo-base}-{tarea}-{versión}`, por ejemplo `acme/llama3-soporte-tecnico-v2`, con el modelo base referenciado en la `model_card` y el `adapter_config.json`. Descargar y usar el adaptador requiere solo el

adaptador y el modelo base original: `PeftModel.from_pretrained(base_model, "acme/llama3-soporte-tecnico-v2")`.

Los **snapshots de producción** son modelos fusionados listos para despliegue: el resultado de `model.merge_and_unload()` que funde el adaptador LoRA con el modelo base en un único conjunto de pesos. Los snapshots representan el artefacto de producción final que se carga directamente en vLLM o se convierte a GGUF para distribución con Ollama. Se versionan con tags semánticos en el registry (`v1.0`, `v1.1-hotfix`, `v2.0`) y se almacenan en el object storage del registry como archivos SafeTensors. Solo los snapshots que han pasado todos los gates de validación (golden dataset + load test) se promueven a "Production" en el sistema de stages del registry; los que no pasan se marcan como "Rejected" con las razones del fallo documentadas.

La **metadata de linaje** es el componente que conecta el snapshot de producción con todos los artefactos que lo producen. Cada versión del modelo en el registry debe incluir: SHA del repositorio Git del código de entrenamiento en el commit exacto usado para ese run, SHA del archivo de dataset (o la versión del dataset en Hugging Face Datasets si es público), versión exacta del modelo base (incluyendo el commit SHA del repositorio de Hugging Face del modelo base), hiperparámetros completos del entrenamiento (rank de LoRA, learning rate, número de epochs), y métricas de evaluación en el golden dataset con el score obtenido. Esta información permite reconstruir completamente cómo se llegó a cada versión y es el fundamento del sistema de ML auditable.

Estrategias de versionado por tipo de artefacto

- **Checkpoints:** guardar cada 100-500 steps; retener solo los N mejores según la métrica de validación; incluyen estado del optimizador (son 2-3x más grandes que el adaptador solo); para reanudación de entrenamiento, no para distribución.
- **Adaptadores LoRA:** convención `{org}/{base_model}-{tarea}-{versión}` en Hugging Face Hub; metadata con SHA del modelo base, dataset y hiperparámetros; versionado independiente del modelo base.
- **Snapshots de producción:** tags semánticos (`v1.0`, `v2.0`); solo snapshots que pasan los gates de validación; almacenados en object storage del registry como SafeTensors o GGUF.
- **Metadata de linaje:** SHA del código de entrenamiento, SHA del dataset, versión del modelo base, hiperparámetros, métricas en el golden set; permite reconstruir el historial completo de cada versión.

- **Git para configuraciones:** Modelfiles de Ollama, configs de Axolotl, scripts de evaluación y definiciones de datasets se versionan en Git normal; los archivos de pesos (grandes) en Git-LFS o en el registry de modelos.

Nota del Arquitecto: La regla mínima de versionado de modelos es versionar siempre la tupla (modelo base + adaptador + dataset) como una unidad. Si cambias cualquiera de los tres —nuevo modelo base, nuevos datos de entrenamiento, mismos datos pero procesados de forma diferente— el resultado es una nueva versión, no una actualización de la versión anterior. Esta granularidad en el versionado es lo que permite comparar versiones de forma significativa y hacer rollback con confianza.

El versionado correcto de artefactos de modelo es el fundamento sobre el que se construyen los procesos de validación pre-despliegue y rollback. La sección siguiente presenta los mecanismos de validación que determinan si una nueva versión del modelo está lista para producción.

Módulo 8 – Capítulo 09 – Sección 03

Validación antes del despliegue: benchmarks de calidad y de performance

Un nuevo modelo nunca se despliega directamente en producción sin pasar por un proceso de validación que verifica dos dimensiones independientes: la calidad funcional (¿el modelo genera las respuestas correctas?) y el rendimiento de infraestructura (¿el modelo cumple los SLOs de latencia y throughput bajo la carga de producción?). La validación de cualquiera de las dos dimensiones por separado es insuficiente: un modelo con calidad perfecta que no cumple los SLOs de latencia destruye la experiencia de usuario; un modelo que cumple los SLOs de latencia pero tiene regresión en calidad introduce errores silenciosos en el producto que pueden tardar días en detectarse.

El **golden dataset** es el componente central de la validación de calidad. Este conjunto de 100-1.000 ejemplos representativos de los casos de uso del producto —construido y mantenido con el mismo cuidado que los tests de integración de software— se evalúa en cada versión candidata del modelo antes del despliegue. Los ejemplos cubren los flujos más críticos del producto, los edge cases conocidos que han producido fallos en el pasado, y las categorías de uso más frecuentes según los logs de producción. Las métricas de evaluación varían según la tarea: exact match para respuestas de opción múltiple o clasificación, ROUGE-L o BERTScore para resúmenes, y LLM-as-judge para respuestas de forma libre donde la calidad no puede medirse con métricas automáticas. El score en el golden dataset es el gate de calidad: si la nueva versión del modelo tiene una degradación superior al umbral definido por el equipo de producto (típicamente 2-5%) respecto a la versión actual de producción, el despliegue se bloquea automáticamente.

La **evaluación LLM-as-judge** complementa las métricas automáticas para respuestas abiertas. El proceso usa un modelo más potente (GPT-4o, Claude Opus) como juez que evalúa pares `(input, expected_output, actual_output)` con un prompt de evaluación estructurado que define los criterios de calidad relevantes para la tarea (precisión, completitud, formato, tono) y produce un score numérico entre 0 y 1 con una justificación textual. Este proceso es automatizable en CI/CD con un umbral de score mínimo configurado: el pipeline de GitHub

Actions puede ejecutar la evaluación LLM-as-judge en el golden dataset como parte del proceso de PR del adaptador LoRA o del snapshot de producción, fallando el PR si el score promedio cae del umbral. La clave es que el prompt de evaluación del juez sea específico para la tarea y validado manualmente antes de automatizarlo.

Los **benchmarks de rendimiento** pre-despliegue deben replicar el patrón de tráfico real de producción: longitud de prompts, distribución de output tokens, y número de requests concurrentes basados en los percentiles reales de producción, no en los mejores casos. El script `benchmark_throughput.py` de vLLM permite configurar estos parámetros y genera un reporte de TTFT P50/P95/P99 y throughput bajo distintos niveles de carga. Este benchmark debe ejecutarse en un entorno de staging con **hardware idéntico a producción** durante al menos 30 minutos bajo la carga P95 esperada: los problemas de degradación de rendimiento bajo carga sostenida (memory leaks, fragmentación del KV cache, scheduling degradation) solo aparecen en runs de duración suficiente.

Los **smoke tests de integración** completan el conjunto de validaciones pre-despliegue: 5-10 requests de ejemplo que cubren los flujos más críticos del producto, ejecutadas contra el modelo candidato en staging, verificando que el modelo responde al formato correcto de request, que el streaming funciona, que los stop tokens se procesan correctamente y que el model name en la respuesta es el esperado. Son la primera línea de defensa: si los smoke tests fallan, no hay razón para ejecutar el golden dataset completo.

Componentes del pipeline de validación previa al despliegue

- **Golden dataset:** 100-1.000 ejemplos representativos de casos de uso reales; métricas exactas o LLM-as-judge según la tarea; gate de calidad con umbral de degradación máxima aceptable.
- **LLM-as-judge evaluation:** modelo más potente como árbitro de calidad para respuestas abiertas; score numérico 0-1 con justificación; automatizable en CI/CD con umbral configurado como gate de PR.
- **Load test pre-despliegue:** hardware idéntico a producción; parámetros de tráfico basados en percentiles reales (longitud de prompts, output, concurrencia); duración mínima de 30 minutos; verificar TTFT-P95 y TBT-P95 contra los SLOs definidos.
- **Smoke tests de integración:** 5-10 requests de flujos críticos; verificación de formato, streaming, stop tokens y model name en la respuesta; primera línea de defensa antes del golden dataset completo.

- **Canary validation:** 5-10% del tráfico real al modelo candidato durante 24-48 horas antes del despliegue completo; comparación de métricas de negocio (thumbs up/down, tasa de completación); el AB test en producción real no puede ser reemplazado por evaluación offline.

Nota del Arquitecto: *El golden dataset mantenido con cuidado y el load test representativo son los dos gates que previenen el 90% de los incidentes de producción relacionados con actualizaciones de modelos. El golden dataset detecta regresiones de calidad; el load test detecta problemas de rendimiento. Omitir cualquiera de los dos es aceptar riesgo de incidente que podría prevenirse con 2-4 horas de trabajo de validación.*

La validación pre-despliegue rigurosa es lo que hace que las actualizaciones de modelos sean operaciones predecibles en lugar de eventos de riesgo. La sección siguiente detalla las estrategias de despliegue progresivo y rollback que completan el ciclo de actualización de modelos en producción.

Módulo 8 – Capítulo 09 – Sección 04

Actualizaciones y rollback: estrategias para reemplazar modelos en producción

El despliegue de una nueva versión de modelo en producción es una operación de mayor riesgo técnico que el despliegue de la mayoría del software convencional, por una razón específica: los modelos de lenguaje pueden exhibir regresiones de calidad sutiles que no se detectan en métricas de infraestructura estándar (latencia, error rate HTTP) sino en métricas de comportamiento (calidad de respuesta, adherencia a instrucciones, manejo de edge cases) que requieren tiempo de observación bajo tráfico real para manifestarse. Un modelo que pasa todos los gates de validación en staging puede mostrar problemas inesperados en producción con el tráfico real de usuarios, que es más diverso y menos predecible que cualquier golden dataset.

El **canary deployment** es el estándar de la industria para mitigar este riesgo: en lugar de reemplazar el modelo actual de producción directamente, se configura un servidor de inferencia paralelo con el nuevo modelo y se enruta un porcentaje pequeño del tráfico (comenzando con 1-5%) mientras se monitorean simultáneamente las métricas de infraestructura (TTFT P95, error rate) y las métricas de producto (engagement, thumbs up/down de usuarios, tasa de completación del flujo). Si ambas categorías de métricas son iguales o mejores que el modelo actual, el porcentaje se incrementa gradualmente con periodos de observación entre cada incremento: 5% → 10% → 25% → 50% → 100%. Este proceso puede tomar 24-72 horas para completarse con suficiente confianza estadística.

En Kubernetes, el canary deployment se implementa manteniendo dos Deployments del mismo servicio: uno con la versión actual del modelo (v1) y uno con la nueva versión (v2), con el mismo Service y el tráfico distribuido mediante labels y pesos en el Ingress, o usando herramientas como Argo Rollouts o Flagger que automatizan el proceso de incremento basado en análisis de métricas. Argo Rollouts con el análisis configurado sobre las métricas de Prometheus puede promover automáticamente el canary si las métricas de TTFT y engagement superan los umbrales definidos, o hacer rollback automático si caen por debajo.

Esta automatización elimina la dependencia de intervención manual durante el proceso de rollout y reduce el tiempo de respuesta ante un problema detectado de minutos a segundos.

El **shadow mode** (o dark launch) es una variante más conservadora del canary: el nuevo modelo recibe una copia del tráfico de producción pero sus respuestas no se envían a los usuarios; en su lugar, se registran en un sistema de logging paralelo para comparación offline. Shadow mode es especialmente útil para validar modelos con cambios arquitectónicos significativos (nuevo modelo base, cambio de cuantización) donde la incertidumbre es alta y el riesgo de mostrar respuestas degradadas a usuarios es inaceptable. El costo es que se requiere doble capacidad de GPU durante el periodo de shadow mode y que los problemas que solo aparecen bajo carga real (no bajo tráfico duplicado en shadow) no son detectables.

El **rollback** debe ser una operación que cualquier ingeniero de guardia pueda ejecutar en menos de 5 minutos con un único comando, sin necesitar el conocimiento especializado del equipo de ML. En Kubernetes con un canary deployment activo, `kubectl argo rollouts undo <rollout-name>` o `kubectl rollout undo deployment/<nombre>` es suficiente para revertir al modelo anterior en producción en segundos. El requisito de que el rollback sea simple y rápido no es solo una buena práctica —es un criterio de diseño de la arquitectura de despliegue: si el rollback requiere más de dos pasos o más de 5 minutos, el diseño necesita ser simplificado antes del siguiente release.

Estrategias de actualización de modelos

- **Canary progresivo:** 1% → 5% → 10% → 25% → 50% → 100% con periodos de observación; análisis automático de métricas con Argo Rollouts; rollback automático si las métricas caen del umbral.
- **Blue-Green deployment:** dos entornos completos activos simultáneamente; rollback instantáneo redirigiendo el Service al entorno anterior; requiere 2x la capacidad de GPU durante la transición.
- **Shadow mode (dark launch):** el nuevo modelo recibe copia del tráfico pero sus respuestas no llegan a usuarios; comparación offline; costo de 2x capacidad; para cambios arquitectónicos de alto riesgo.
- **Feature flags para modelos:** control granular del segmento de usuarios que recibe cada versión del modelo; permite AB tests por plan de usuario, idioma o tipo de uso; más flexible que el canary a nivel de tráfico total.
- **Runbook de rollback:** procedimiento exacto documentado (qué comando, quién tiene permisos, cómo verificar el éxito); ejecutable por cualquier persona de guardia en <5

minutos.

Nota del Arquitecto: El rollback rápido y sencillo es el safety net que permite desplegar con confianza. Si sé que puedo revertir en 2 minutos con un comando, puedo avanzar el canary con mayor velocidad y asumir más riesgo calculado en el proceso. Si el rollback toma 20 minutos y requiere coordinación de 3 personas, el equipo será conservador en los despliegues y el ciclo de mejora se ralentizará. El rollback no es el plan B; es la condición que hace posible el plan A.

La capacidad de desplegar con confianza y hacer rollback rápidamente es el sello de un sistema de ML maduro. La sección siguiente complementa el ciclo de vida con los controles de seguridad necesarios para proteger los artefactos de modelo como los activos de alto valor que son.

Módulo 8 – Capítulo 09 – Sección 05

Seguridad de modelos: verificación de checksums y protección de pesos propietarios

Los artefactos de modelo de ML tienen dos vectores de seguridad que no existen en el software convencional de la misma forma: la integridad (garantizar que los pesos descargados son exactamente los originales y no han sido alterados) y la confidencialidad (proteger pesos propietarios obtenidos mediante fine-tuning con datos exclusivos de la organización). Ambos vectores son amenazas reales, no hipotéticas: en 2023 se detectaron modelos con nombres similares a modelos populares en Hugging Face Hub con código malicioso embebido; los pesos fine-tuneados sobre datos propietarios de clientes son activos de alto valor que representan semanas de ingeniería de datos y miles de dólares en compute.

La **verificación de integridad** comienza con entender el formato de los archivos de pesos. Los archivos PyTorch `.bin` usan `pickle.load()` para deserializar, y `pickle` puede ejecutar código Python arbitrario embebido en el archivo —es el mecanismo por el que los ataques de supply chain contra modelos ML son posibles. Un actor malicioso que publique en Hugging Face Hub un modelo con nombre similar a Llama 3.1 8B (`meta-Ilama/Llama-3.1-8B-Instruct` con una L minúscula en lugar de una i mayúscula) puede incluir código malicioso que se ejecuta en el momento de la deserialización con `from_pretrained()`. Los archivos SafeTensors (`.safetensors`), diseñados específicamente para este problema, solo pueden contener tensores numéricos en un formato verificable; no pueden contener código ejecutable. Migrar todos los modelos internos al formato SafeTensors es una medida de seguridad con costo de implementación bajo y beneficio de seguridad alto.

La **verificación de checksums** complementa la protección de formato. Hugging Face Hub calcula y almacena hashes SHA-256 de cada archivo de modelo en los metadatos del repositorio; el downloader `huggingface_hub.snapshot_download()` verifica automáticamente la integridad de cada archivo al completar la descarga. Para archivos GGUF descargados de fuentes externas (registros de terceros, links directos de repositorios GitHub), la verificación manual con `sha256sum modelo.gguf` y comparación con el hash publicado por el autor es la primera línea de defensa. ModelScan de Protect AI es una herramienta de análisis estático

que escanea archivos `.pkl`, `.pt` y `.bin` en busca de patrones maliciosos conocidos, integrable en CI/CD como gate previo a la carga de modelos en el registry interno.

La **protección de pesos propietarios** requiere controles en tránsito y en reposo. Los modelos fine-tuneados sobre datos sensibles deben almacenarse con cifrado en reposo: S3 SSE-KMS para almacenamiento en AWS (con las claves de cifrado gestionadas en AWS KMS separado del storage), BitLocker o dm-crypt para NVMe local en servidores on-premise. En tránsito, TLS 1.3 para todas las transferencias de modelos. El control de acceso granular es igualmente crítico: en producción, únicamente los sistemas de serving deben tener acceso de lectura a los archivos de pesos; los ingenieros deben acceder a las capacidades del modelo exclusivamente a través de la API de inferencia, nunca directamente a los archivos de pesos. Esta separación de acceso evita la exfiltración accidental o deliberada de pesos propietarios.

Para organizaciones con modelos de muy alto valor, el **watermarking de pesos** (inyección de marcas de agua en los pesos del modelo durante el proceso de fusión) permite detectar si un modelo propietario ha sido exfiltrado y redistribuido por un tercero: las marcas de agua sobreviven el proceso de conversión de formato (SafeTensors → GGUF) y pueden verificarse en cualquier modelo candidato con la clave de watermarking de la organización.

Controles de seguridad para modelos ML

- **SafeTensors en lugar de PyTorch .bin:** SafeTensors no puede contener código ejecutable; migrar con `from safetensors.torch import save_file`; aplicar a todos los modelos internos y de terceros cargados en producción.
- **Verificación de checksums:** `huggingface_hub.snapshot_download()` verifica automáticamente; para archivos externos, comparar SHA-256 con el hash publicado por el autor del modelo.
- **ModelScan:** análisis estático de archivos de modelos en busca de patrones maliciosos; integrable en CI/CD como gate previo a la carga en el registry.
- **Cifrado en tránsito y en reposo:** TLS 1.3 para transferencias; S3 SSE-KMS o cifrado de disco para almacenamiento; claves en KMS separado del storage.
- **Control de acceso basado en roles (RBAC):** solo los sistemas de serving tienen acceso de lectura a los pesos en producción; los ingenieros acceden vía API de inferencia; auditoría de todos los accesos de descarga.

Nota del Arquitecto: El vector de ataque que más frecuentemente se ignora en despliegues de modelos locales es el accidental: un ingeniero que copia los pesos a su

laptop para "hacer pruebas locales" sin darse cuenta de que esos pesos contienen, efectivamente, el modelo de embeddings de los documentos confidenciales de los clientes en sus pesos fine-tuneados. El control de acceso basado en roles que limita el acceso directo a los archivos de pesos no es paranoia; es la misma práctica que aplicamos a los dumps de base de datos con datos de clientes.

La seguridad de los artefactos de modelo es parte integral del governance del ciclo de vida de ML. La sección de cierre consolida el capítulo estableciendo los modelos como artefactos de software que requieren el mismo nivel de gobierno.

Módulo 8 – Capítulo 09 – Sección 06

Cierre: los modelos son artefactos de software y requieren el mismo gobierno

La analogía más útil para comunicar el gobierno de modelos a una organización que viene del mundo del software es directa: los modelos de lenguaje son artefactos de ingeniería con ciclo de vida, dependencias, vulnerabilidades y requisitos de governance análogos a los del software de producción. Un modelo sin versionado es como código sin git: funciona hasta que necesitas entender qué cambió entre dos versiones y no tienes forma de saberlo. Un modelo sin validación pre-despliegue es como código sin tests: funciona hasta que no funciona, y entonces no sabes por qué. Un modelo sin proceso de rollback es como código sin capacidad de revertir commits: cada despliegue es un punto de no retorno.

La madurez del ecosistema de herramientas de gobierno de modelos ha reducido el costo de implementar estas prácticas a un nivel accesible para cualquier equipo. Hugging Face Hub o MLflow para el registry de modelos, un golden dataset mantenido en el repositorio Git del proyecto, GitHub Actions o Jenkins para ejecutar la validación automatizada, Argo Rollouts para el despliegue progresivo canary, y `kubectl rollout undo` para el rollback: estas son herramientas ampliamente disponibles y bien documentadas que, combinadas con los procesos presentados en este capítulo, constituyen el mínimo viable de gobierno de modelos para un sistema de producción.

El gobierno de modelos también tiene dimensiones de compliance que están siendo codificadas en regulación: el EU AI Act para sistemas de alto riesgo requiere documentación del proceso de entrenamiento, de las métricas de evaluación de sesgos y de las restricciones de uso; el Executive Order de IA de EEUU establece requisitos de transparencia para modelos de gran capacidad. Los equipos que implementan el governance desde el principio —model cards documentados, linaje de datos trazable, evaluaciones de sesgo en el golden dataset, restricciones de uso documentadas— estarán mejor posicionados cuando estos requisitos regulatorios se apliquen a sus sistemas. Los que los implementen como afterthought tendrán que documentar retroactivamente sistemas opcos que han operado durante meses sin trazabilidad.

El CI/CD para modelos —automatizar el pipeline desde el commit del código de entrenamiento hasta el despliegue en producción pasando por todos los gates de validación— es el estado del arte del MLOps que los equipos más maduros están implementando en 2025. Un pipeline típico integra: validación del dataset al hacer PR del script de preparación de datos, entrenamiento automático con Axolotl en una instancia cloud Spot al fusionar al main branch del adaptador, evaluación con lm-evaluation-harness en el golden set como CI step, push al Hugging Face Hub solo si se superan los umbrales de calidad, y trigger automático del canary deployment en Kubernetes al publicar una nueva versión en el registry. Este pipeline convierte el fine-tuning de una operación manual en un proceso de ingeniería reproducible y auditable.

Idea central

La diferencia entre un proyecto de ML experimental y un sistema de ML en producción es precisamente el governance: versionado, validación, despliegue controlado y auditoría no son burocracia sino la ingeniería que permite operar modelos con confianza a escala.

"Software engineering is what happens to programming when you add time and other programmers." — Russ Cox, ingeniero en Google y contribuidor principal a Go, recordando que la gestión del ciclo de vida de modelos ML requiere exactamente las mismas disciplinas de ingeniería de software que cualquier otro sistema de producción que evoluciona en el tiempo.